Karlsruher Institut für Technologie

Institut für Programmstrukturen und Datenorganisation

Prof. Dr.-Ing. Gregor Snelting
gregor.snelting@kit.edu

Prof. Dr. Ralf H. Reussner
reussner@kit.edu

Klausur Programmierparadigmen — Beispiellösung

WS21/22, 08. April 2022, 12:00 – 14:00 Uhr

Zugelassene Hilfsmittel: Papierbasierte Quellen (Vorlesungsfolien, Übungsblätter, eigene Aufzeichnungen, Bücher, ...)
Die Verwendung von elektronischen Geräten ist verboten.

Bearbeitungszeit: 120 min

Aufgabe 1 (Haskell: Dateisystem) [13 Punkte]

Mit dem folgenden Datentypen modellieren wir ein einfaches Dateisystem:

```
data Entry = File String
           | Dir [(String, Entry)]
           deriving (Show)

type Path = [String]
```

Ein Dateisystemeintrag (`Entry`) ist entweder eine Datei (`File`) mit einem String als Inhalt oder ein Verzeichnis (`Dir`). Ein Verzeichnis enthält eine Liste von Paaren aus Datei-/Verzeichnisnamen und den zugehörigen Einträgen. In dieser Liste kommt jeder Name höchstens einmal vor. Außerdem ist ein Dateisystempfad (`Path`) als Liste von Strings definiert.

Hier sehen Sie ein Beispiel-Dateisystem:

```
example = Dir [("test", File "Hello World!"),
               ("bin", Dir [("ls", File "ls ..."),
                            ("cat", File "cat ..."),
                            ("du", File "du -s ...")]),
               ("home", Dir [("propa", Dir [("id", File "id x = x")])])]
```

Bewertungshinweis: In dieser Aufgabe wird verlangt, dass Sie explizit mittels `error` Fehlermeldungen ausgeben. Eine kurze Meldung, die die Art des Fehlers beschreibt, reicht aus. Haskell-interne Fehlermeldungen (z. B. fehlgeschlagenes Pattern Matching) werden aber *nicht* gewertet.

(a) Implementieren Sie eine Funktion [2 Punkte]

```
ls :: Entry -> [String]
```

Der Aufruf `ls (Dir entries)` soll die Namen aller Einträge im übergebenen Verzeichnis zurückgeben. Der Aufruf `ls (File _)` soll mittels der Funktion `error` mit einer sinnvollen Fehlermeldung abbrechen.

Beispiel: `ls example` $\Longrightarrow$ `["test", "bin", "home"]`

(b) Implementieren Sie eine Funktion [3 Punkte]

```
du :: Entry -> Int
```

Sie soll die Summe der Längen aller Dateien im übergebenen Dateisystem berechnen.

Beispiel: `du example` $\Longrightarrow$ `42`

(c) Implementieren Sie eine Funktion [6 Punkte]

```
getEntry :: Entry -> Path -> Entry
```

Der Aufruf `getEntry root path` soll den Eintrag zurückgeben, der relativ zum Eintrag `root` am Pfad `path` liegt. Bei Fehlern soll `getEntry` mittels der Funktion `error` mit einer sinnvollen Fehlermeldung abbrechen.

Beispiele:

`getEntry example ["bin", "ls"]` $\Longrightarrow$ `File "ls ..."`

`getEntry example ["home", "propa"]` $\Longrightarrow$ `Dir [("id",File "id x = x")]`

(d) Implementieren Sie eine Funktion [2 Punkte]

```
cat :: Entry -> Path -> String
```

Der Aufruf `cat root path` soll die Datei ausgeben, die relativ zum Eintrag `root` am Pfad `path` liegt. Bei Fehlern soll `cat` mittels der Funktion `error` mit einer sinnvollen Fehlermeldung abbrechen.

Hinweis: Ein **`case`**-Ausdruck bietet sich an.

Beispiel: `cat example ["home", "propa", "id"]` $\Longrightarrow$ `"id x = x"`

Beispiellösung:

```
ls :: Entry -> [String]
ls (File _)      = error "not a directory"
ls (Dir entries) = map fst entries

du :: Entry -> Int
du (File f) = length f
du (Dir entries) = sum (map (du . snd) entries)

getEntry :: Entry -> Path -> Entry
getEntry root                          []       = root
getEntry (File _)                      _        = error "not a directory"
getEntry (Dir [])                      _        = error "entry not found"
getEntry (Dir ((name,entry):entries)) (p:path) =
  if name == p then getEntry entry         path
               else getEntry (Dir entries) (p:path)

-- alternativ mit Hilfsfunktion ...
getEntry' :: Entry -> Path -> Entry
getEntry' root          []       = root
getEntry' (File _)      _        = error "not a directory"
getEntry' (Dir entries) (p:path) = getEntry' (findInDir entries) path
  where
    findInDir [] = error "entry not found"
    findInDir ((name, entry):entries) =
      if name == p then entry
                   else findInDir entries

-- ... oder vielleicht mit lookup
getEntry'' :: Entry -> Path -> Entry
getEntry'' root          []       = root
getEntry'' (File _)      _        = error "not a directory"
getEntry'' (Dir entries) (p:path) = case lookup p entries of
                                      Nothing -> error "entry not found"
                                      Just e  -> getEntry'' e path

cat :: Entry -> Path -> String
cat root path = case getEntry root path of
                  File contents -> contents
                  Dir  _        -> error "is a directory"
```

Aufgabe 2 (Haskell: Passwort-Cracking) [17 Punkte]

Beim Passwort-Cracking sucht ein Angreifer einen String, der nach Anwendung einer Hashfunktion den gleichen Hash erzeugt wie das Passwort des Benutzers.

Aufgrund der Art und Weise, wie ein Großteil der Benutzer ihr Passwort wählt, bietet sich folgende Vorgehensweise an: Der Angreifer generiert ein Wörterbuch von Passwörtern, die er der Reihe nach durchprobiert. Zum Generieren dieses Wörterbuchs startet er mit einem Wörterbuch häufiger Passwörter und wendet dann auf diese Wörter Modifikationen an (z. B. Ersetzen von Buchstaben).

In dieser Aufgabe modellieren wir Wörterbücher und Modifikations-Funktionen folgendermaßen:

```
-- Ein Wörterbuch ist eine Liste von Strings
type Dict = [String]
-- Eine Modifikations-Funktion nimmt einen String
-- und gibt eine Liste von modifizierten Strings zurück
type Mod = String -> [String]
```

Hashes werden durch den Datentyp `Hash` repräsentiert, der die Typklasse `Eq` implementiert. Eine Hashfunktion hat daher den Typ `String -> Hash`.

(a) Implementieren Sie die Funktion [3 Punkte]

```
replaceChar :: Char -> Char -> Mod
```

`replaceChar c1 c2` ist eine Modifikations-Funktion, die für einen übergebenen String `s` ein einzelnes modifiziertes Wort zurückgibt, welches aus `s` durch Ersetzen aller Vorkommen von `c1` durch `c2` entsteht.
Beispiel: `(replaceChar 'A' '4') "ADAC"` $\Rightarrow^*$ `["4D4C"]`

(b) Implementieren Sie die Funktion [3 Punkte]

```
insertEverywhere :: Char -> Mod
```

`insertEverywhere c` ist eine Modifikations-Funktion, die für einen übergebenen String `s` die Strings zurückgibt, die aus `s` durch Einfügen von `c` an allen möglichen Stellen entstehen.
Hinweis: Die Funktionen `take` und `drop` könnten hilfreich sein.
Beispiel: `(insertEverywhere '$') "abc"` $\Rightarrow^*$ `["$abc", "a$bc", "ab$c", "abc$"]`

(c) Implementieren Sie die Funktion [5 Punkte]

```
genStep :: Dict -> [Mod] -> Dict
```

`genStep dict mods` gibt ein Wörterbuch aller Wörter aus, die durch Modifizieren eines Worts aus `dict` durch eine Modifikations-Funktion aus `mods` entstehen können.
Dabei kann `dict` eine endliche oder unendliche Liste sein, `mods` ist dagegen stets eine endliche Liste.
Hinweis: List Comprehension könnte nützlich sein.

(d) Implementieren Sie die Funktion [3 Punkte]

```
gen :: Dict -> [Mod] -> Dict
```

`gen dict mods` gibt ein unendliches Wörterbuch aller durch den Angreifer durchzuprobierender Wörter aus. Dabei enthält das Ergebnis zuerst die Wörter aus `dict`, dann die durch einen `genStep` daraus entstehenden Wörter, danach die durch einen weiteren `genStep` entstehenden Wörter usw.

(e) Implementieren Sie die Funktion [2 Punkte]

```
findPassword :: (String -> Hash) -> Dict -> Hash -> String
```

`findPassword hf dict h` gibt das erste Wort aus `dict` zurück, für das die Anwendung der Hashfunktion `hf` den Hash `h` ergibt. Hierbei ist `dict` immer eine unendliche Liste.

(f) Gegeben sei die Funktion [1 Punkt]

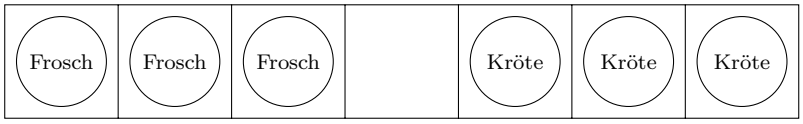
```
crack x y z = findPassword x (gen y z)
```

welche das Passwort-Cracking durchführt. Geben Sie den allgemeinsten Typen von `crack` an.

Beispiellösung:

```
replaceChar :: Char -> Char -> Mod
replaceChar c1 c2 s = [map (\c -> if c == c1 then c2 else c) s]

--alternativ:
replaceChar' :: Char -> Char -> Mod
replaceChar' c1 c2 cs = [replaceCharHelper cs]
    where replaceCharHelper [] = []
          replaceCharHelper (c:cs) | c == c1 = c2:replaceCharHelper cs
                                   | otherwise = c:replaceCharHelper cs

insertEverywhere :: Char -> Mod
insertEverywhere c s = map (\n -> take n s ++ c : drop n s) [0..length s]

-- Bemerkung: Die Reihenfolge von "s <- dict" und "mod <- mods" ist wichtig:
-- Wenn dict unendlich ist, werden für ein Wort aus dem dict
-- alle endlich vielen mods angewendet,
-- und dann mit dem nächsten Wort weitergemacht.
-- Dadurch kommt jedes Wort aus dem dict und jede Modifikationsfunktion dran.
-- Wäre es "mod <- mods, s <- dict", würde ein Element aus mods
-- auf alle Wörter angewendet, bevor es mit dem nächsten Element weitergeht.
-- Wenn dict unendlich ist, kommt daher nur das Kopf-Element von mods dran.
-- Das gleiche Problem ergibt sich bei Definitionen der Form
-- "genStep dict mods = concatMap (\m->concatMap m dict) mods".
genStep :: Dict -> [Mod] -> Dict
genStep dict mods = [s2 | s <- dict, mod <- mods, s2 <- mod s]

gen :: Dict -> [Mod] -> Dict
gen dict mods = dict ++ gen (genStep dict mods) mods

findPassword :: (String -> Hash) -> Dict -> Hash -> String
findPassword hf dict h = head (filter (\s -> hf s == h) dict)

crack :: (String -> Hash) -> Dict -> [Mod] -> Hash -> String
```

Aufgabe 3 (Prolog: Frösche und Kröten) [16 Punkte]



`[f, f, f, l, k, k, k]`

Startzustand für $n = 3$, rechts als Prolog-Liste codiert.

In dieser Aufgabe sollen Sie das „Frösche und Kröten"-Rätsel in Prolog lösen. Zu Beginn stehen sich auf einem Spielfeld von $2n + 1$ Feldern links n Frösche und rechts n Kröten mit einem freien Feld zwischen ihnen gegenüber. Ziel ist, dass Frösche und Kröten die Plätze tauschen. Es gelten folgende Regeln:

- Frösche bewegen sich nur nach rechts, Kröten nur nach links.
- Ein Tier kann entweder auf ein freies Feld neben sich ziehen, oder über *ein* Tier der anderen Art auf ein freies Feld springen.

Wir modellieren das Spielfeld als Prolog-Liste mit den Elementen `f` (Frosch), `k` (Kröte) oder `l` (leer). Dann sind also folgende Züge möglich, die in Regeln für das Prädikat `rule` encodiert sind:

```
  [...,f,l,...]  ⟶  [...,l,f,...]    rule([f, l], [l, f]).
  [...,l,k,...]  ⟶  [...,k,l,...]    rule([l, k], [k, l]).
[...,f,k,l,...]  ⟶  [...,l,k,f,...]  rule([f, k, l], [l, k, f]).
[...,l,f,k,...]  ⟶  [...,k,f,l,...]  rule([l, f, k], [k, f, l]).
```

Das Prädikat `rule` betrachtet also nur die Felder, auf denen beim Zug etwas passiert. Das Prädikat `step` soll nun den Effekt eines Zuges auf das gesamte Spielfeld modellieren.

(a) Definieren Sie nun das Prädikat `step(Before, After)`. Es soll genau dann erfüllbar sein, wenn das Spielfeld mit einem Zug vom Zustand `Before` in den Zustand `After` kommt. Dabei sei `Before` immer instanziiert. [4 Punkte]
Hinweis: Probieren Sie für jede Teilliste von `Before`, ob eine durch `rule` definierte Regel anwendbar ist.

(b) Definieren Sie das Prädikat `game(Start, End, Moves)`. Es soll genau dann erfüllbar sein, wenn man vom Zustand `Start` über die Zwischenzustände `Moves` in den Zustand `End` gelangt. Dabei sei `Start` immer instanziiert. Es bleibt Ihrer Implementierung überlassen, ob `Start` und/oder `End` Teil von `Moves` sind. [3 Punkte]
Hinweis: Aufgrund der Struktur des Spiels gibt es nur endliche Zugfolgen. Sie brauchen sich daher nicht um Zyklenerkennung zu kümmern.

Um den Start- und Endzustand des Rätsels zu generieren, brauchen wir zunächst zwei Hilfsprädikate:

(c) Definieren Sie das Prädikat `alleq(X, Xs)`. Es soll genau dann erfüllbar sein, wenn alle Elemente der Liste `Xs` gleich `X` (d. h. mit `X` unifizierbar) sind. [2 Punkte]

(d) Definieren Sie das Prädikat `replicate(N, X, Xs)`. Es soll genau dann erfüllbar sein, wenn `Xs` die Liste ist, die aus `N` Wiederholungen des Elements `X` besteht. [2 Punkte]
Hinweis: Sie können das vordefinierte Prädikat `length(Xs, N)` benutzen, um in `Xs` eine Liste von `N` uninstanziierten Elementen zu generieren.

(e) Definieren Sie nun das Prädikat `start(N, Field)`. Es soll genau dann erfüllbar sein, wenn `Field` die Startkonfiguration des Rätsels mit `N` Fröschen und Kröten ist, also die Liste [3 Punkte]

$$[\overbrace{\texttt{f},\ldots,\texttt{f}}^{\texttt{N mal}},\texttt{l},\overbrace{\texttt{k},\ldots,\texttt{k}}^{\texttt{N mal}}]$$

Ab jetzt sei analog zu `start` auch das Prädikat `finish(N, Field)` definiert.

(f) Definieren Sie das Prädikat `solution(N, Moves)`. Es soll genau dann erfüllbar sein, wenn `Moves` eine Lösung des Rätsels mit `N` Fröschen und Kröten darstellt. [2 Punkte]

Beispiellösung:

```
/* nicht verlangt */
rule([f, l], [l, f]).
rule([l, k], [k, l]).
rule([f, k, l], [l, k, f]).
rule([l, f, k], [k, f, l]).

step(Before, After) :-
    append(A, BC, Before),
    append(B, C, BC),
    rule(B, B1),
    append(B1, C, BC1),
    append(A, BC1, After).

game(Start, Start, []).
game(Start, End, [Next|Rest]) :-
    step(Start, Next),
    game(Next, End, Rest).

alleq(_, []).
alleq(X, [X|Xs]) :-
    alleq(X, Xs).

replicate(N, X, Xs) :-
    length(Xs, N),
    alleq(X, Xs).

start(N, Field) :-
    replicate(N, f, Frogs),
    replicate(N, k, Toads),
    append(Frogs, [l|Toads], Field).

/* nicht verlangt */
finish(N, Field) :-
    replicate(N, f, Frogs),
    replicate(N, k, Toads),
    append(Toads, [l|Frogs], Field).

solution(N, Moves) :-
    start(N, S), finish(N, F), game(S, F, Moves).
```

Aufgabe 4 (Reduktionsgraphen im λ-Kalkül) [15 Punkte]

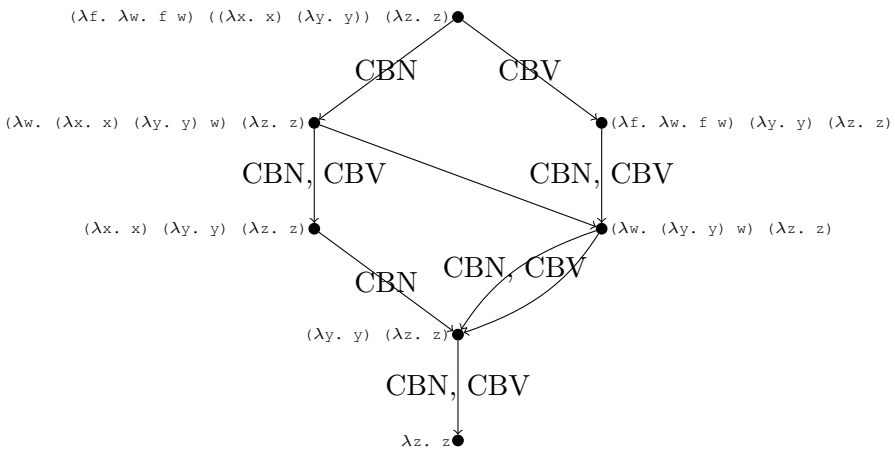
Für einen λ-Term t sei der *Reduktionsgraph* (V_t, E_t) von t wie folgt definiert:

- V_t enthält alle λ-Terme, welche sich durch wiederholte β-Reduktion von t erreichen lassen. Hierbei werden *alle* möglichen Reduktionen der jeweiligen Terme betrachtet. α-äquivalente Terme werden *vom selben Knoten* repräsentiert.
- E_t enthält alle zulässigen β-Reduktionsschritte. Es werden nicht mehrere Schritte zusammengefasst. Unterschiedliche Redexe werden zu unterschiedlichen Kanten.

(a) Zeichnen Sie den Reduktionsgraphen des folgenden λ-Terms: [8 Punkte]

$$t = \texttt{(}\lambda\texttt{f. }\lambda\texttt{w. f w) ((}\lambda\texttt{x. x) (}\lambda\texttt{y. y)) (}\lambda\texttt{z. z)}$$

Beispiellösung:



(b) Markieren Sie mit "CBV" und "CBN" diejenigen Kanten auf dem Reduktionsgraphen von t, welche den von den Auswertungsstrategien Call-by-Value bzw. Call-by-Name ausgewählten Reduktionen entsprechen. [4 Punkte]

(c) Geben Sie λ-Terme t_1, t_2 und t_3 an, welche jeweils die folgenden Eigenschaften haben: [3 Punkte]

i. Der Reduktionsgraph von t_1 enthält keine Kanten.

ii. Der Reduktionsgraph von t_2 enthält einen Zyklus.

iii. Der Reduktionsgraph von t_3 ist zyklenfrei, aber unendlich.

Beispiellösung:

i. λ`z. z`

ii. `(`λ`x. x x) (`λ`x. x x)`

iii. `(`λ`x. x x x) (`λ`x. x x x)`

Aufgabe 5 (Typinferenz) [13 Punkte]

(a) Geben Sie einen allgemeinsten Typ τ_{nil} von [2 Punkte]

$$nil = \lambda \mathtt{c}.\ \lambda \mathtt{n}.\ \mathtt{n}$$

an.

Beispiellösung:

$$\tau_{nil} = \alpha \to \beta \to \beta$$

(b) Geben Sie einen allgemeinsten Typ τ_{cons} von [3 Punkte]

$$cons = \lambda \mathtt{x}.\ \lambda \mathtt{xs}.\ \lambda \mathtt{c}.\ \lambda \mathtt{n}.\ \mathtt{c}\ \mathtt{x}\ \mathtt{xs}$$

an.

Beispiellösung:

$$\tau_{cons} = \alpha \to \beta \to (\alpha \to \beta \to \gamma) \to \delta \to \gamma$$

(c) Zeigen Sie, dass der Term [8 Punkte]

$$\textbf{let}\ \mathtt{nil} = \lambda \mathtt{c}.\ \lambda \mathtt{n}.\ \mathtt{n}\ \textbf{in}\ \mathtt{nil}\ \mathtt{nil}$$

typisierbar ist. Vervollständigen Sie dafür den rechten Teil des folgenden Herleitungsbaum (also nach Substitution), indem Sie die fehlenden Typen A, B, C, D angeben.

$$\textit{Let}\ \dfrac{\textit{Abs}\ \dfrac{\dots}{\vdash \lambda \mathtt{c}.\ \lambda \mathtt{n}.\ \mathtt{n} : \tau_{nil}} \qquad \textit{App}\ \dfrac{\textit{Var}\ \dfrac{\Gamma_{let}(\mathtt{nil}) = A \quad A \succeq B}{\Gamma_{let} \vdash \mathtt{nil} : B} \qquad \textit{Var}\ \dfrac{\Gamma_{let}(\mathtt{nil}) = A \quad A \succeq C}{\Gamma_{let} \vdash \mathtt{nil} : C}}{\Gamma_{let} \vdash \mathtt{nil}\ \mathtt{nil} : D}}{\vdash \textbf{let}\ \mathtt{nil} = \lambda \mathtt{c}.\ \lambda \mathtt{n}.\ \mathtt{n}\ \textbf{in}\ \mathtt{nil}\ \mathtt{nil} : D}$$

Beispiellösung:

$$\begin{aligned}
A &= \forall \alpha.\ \forall \beta.\ \alpha \to \beta \to \beta \\
B &= \alpha_2 \to \alpha_3 \to \alpha_3 \to \alpha_1 \to \alpha_1 \\
C &= \alpha_2 \to \alpha_3 \to \alpha_3 \\
D &= \alpha_1 \to \alpha_1
\end{aligned}$$

Aufgabe 6 (Grundlagen der Parallelprogrammierung) [5 Punkte]

Mithilfe der Monte-Carlo-Methode kann die Kreiszahl (Pi) approximiert werden. Eine mögliche sequentielle Implementierung wird im folgenden Code-Beispiel gezeigt.

Dabei werden Punkte zufällig in einem Quadrat mit dem Flächeninhalt 1 (Einheitsquadrat) platziert (Zeile 15–19). Anschließend wird für jeden Punkt bestimmt, ob er in einem Viertelkreisbogen mit dem Radius 1 liegt (Zeile 3–7). Das Verhältnis der Anzahl der Punkte in dem Viertelkreisbogen zu der Gesamtanzahl der Punkte ist ein Maß für ein Viertel der Kreisfläche und damit ein Viertel von Pi (Zeile 9).

Gehen Sie davon aus, dass die Funktion `approximatePi` nur mit positiven Werten für `number_points` aufgerufen wird. Gehen Sie weiterhin davon aus, dass die Funktion `randomNumber` einen `double` mit zufälligem Wert zwischen 0 und 1 zurückgibt und problemlos parallel aufgerufen werden kann.

```
1  double calculatePointsInCircle(double *values_x, double *values_y, int
      number_points) {
2      int points_in_circle = 0;
3      for (int i = 0; i < number_points; i++) {
4          //check if point is within unit circle
5          double z = values_x[i] * values_x[i] + values_y[i] * values_y[i];
6          if (z <= 1) points_in_circle++;
7      }
8
9      double quarter_of_pi = ((double) points_in_circle) / number_points;
10     return quarter_of_pi;
11 }
12
13 double approximatePi(int number_points) {
14   double values_x[number_points], double values_y[number_points];
15   for (int i = 0; i < number_points; i++) {
16     //place points randomly
17     values_x[i] = randomNumber();
18     values_y[i] = randomNumber();
19   }
20
21   double quarter_of_pi = calculatePointsInCircle(values_x, values_y,
        number_points);
22   double pi = 4.0 * quarter_of_pi;
23   return pi;
24 }
```

(a) Bietet sich für die Parallelisierung von `calculatePointsInCircle` die Verwendung von Daten- oder Task-Parallelismus an? Nennen Sie, welche Art von Parallelismus sich eignet, und begründen Sie Ihre Entscheidung kurz. [2 Punkte]

Beispiellösung: Daten-Parallelismus, da die gleiche Aufgabe – das Überprüfen, ob die Punkte im Viertelkreisbogen liegen – gleichzeitig für verschiedene Daten (= Punkte) ausgeführt werden kann.

(b) Beschreiben Sie abstrakt (ohne Nennung von konkreten Konstrukten), wie Sie `calculatePointsInCircle` parallelisieren würden. Gehen Sie dabei auch auf eventuell notwendige Synchronisationen ein. [3 Punkte]

Beispiellösung: Die Iterationen der Schleife, die jeweils überprüfen, ob ein Punkt innerhalb des Einheitskreises liegt, können parallel ausgeführt werden. Hierbei gibt es keine Synchronisationsbedarfe. Für das Inkrementieren von `points_in_circle` muss synchronisiert werden. Vor dem Berechnen von `quarter_of_pi` in Zeile 9 muss auf das Beenden aller Iterationen gewartet werden.

Aufgabe 7 (MPI) [11 Punkte]

Gegeben ist die untenstehende Methode `foo`, welche eine quadratische Matrix (Parameter `values`) sowie die Anzahl Spalten und Reihen (Parameter `x`) erhält. Die Matrix wird als eindimensionales Array repräsentiert, wobei auf das Element in Reihe i und Spalte j mit

$Element[i, j] = values[i * x + j]$

zugegriffen werden kann. Es wird davon ausgegangen, dass MPI korrekt initialisiert und finalisiert wird und die Anzahl der Elemente in `values` je Dimension der Anzahl Prozesse entspricht.

```
1  void foo(int x, int *values) {
2      int local_values[x];
3      int recv_values[x];
4
5      MPI_Scatter(values, x, MPI_INT, local_values, x, MPI_INT, 0,
           MPI_COMM_WORLD);
6
7      MPI_Alltoall(local_values, 1, MPI_INT, recv_values, 1, MPI_INT,
           MPI_COMM_WORLD);
8
9      MPI_Gather(recv_values, x, MPI_INT, values, x, MPI_INT, 0,
           MPI_COMM_WORLD);
10 }
```

(a) Füllen Sie in den nachfolgenden Tabellen aus, wie die Puffer nach Ausführung der in den Tabellen gegebenen Quelltext-Zeile belegt sind. Nehmen Sie an, dass das Array [6 Punkte]
`values[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15}`
mit `x = 4` übergeben wird und das Programm mit 4 Prozessen bearbeitet wird.

Beispiellösung: Siehe Tabellen.

(a) **`local_values` (Nach Zeile 5)**

Prozess Nr.	**`Inhalt` (Lösung)**
0	0, 1, 2, 3
1	4, 5, 6, 7
2	8, 9, 10, 11
3	12, 13, 14, 15

(b) **`recv_values` (Nach Zeile 7)**

Prozess Nr.	**`Inhalt` (Lösung)**
0	0, 4, 8, 12
1	1, 5, 9, 13
2	2, 6, 10, 14
3	3, 7, 11, 15

(c) **`values` (Nach Zeile 9)**

Prozess Nr.	**`Inhalt` (Lösung)**
0	{0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15}

(b) Beschreiben Sie, welche Funktionalität von `foo` realisiert wird. [2 Punkte]

Beispiellösung: Das `values`-Array wird transponiert.

(c) Auf welchen Prozessen muss `values` nicht initialisiert sein, ohne dass sich das Verhalten des Programms ändert? Begründen Sie ihre Antwort. [2 Punkte]

Beispiellösung: Auf allen außer dem Prozess mit Rank 0, da der `MPI_Scatter`-Aufruf in Zeile 5 den `local_values`-Puffer befüllt, mit dem weitergerechnet wird. Im `MPI_Scatter`-Aufruf wird `rank 0` als `root` festgelegt, sodass `values` nur auf Prozess 0 initialisiert sein muss.

(d) Durch welche kollektive MPI-Operation kann die `MPI_Gather`-Operation in Zeile 9 ersetzt werden, damit das Ergebnis der Berechnung im `values`-Array auf allen Prozessen vorliegt? [1 Punkt]
Hinweis: Es müssen keine Parameter oder deren Belegungen angegeben werden.

Beispiellösung: MPI_Allgather

Aufgabe 8 (Java und Design-By-Contract) [9 Punkte]

Gegeben sei das Fragment der Klasse `Restaurant`. Die Klasse bietet einem Restaurant folgende Funktionalität:

1. Offene Bestellungen speichern (`addOrder`),
2. Preis einer Bestellung in Cent berechnen (`calculatePrice`)
3. Bestellungen löschen, sobald sie ausgeliefert wurden (`removeOrder`)

Da das Restaurant neu eröffnet hat, werden Bestellungen rabattiert: Den Kunden werden nur Hauptgerichte, die mehr als 5€ kosten, berechnet. Trotzdessen wird aus Übersichtsgründen für das Restaurant auch der tatsächliche Preis der offenen Bestellungen erfasst (`priceOfOpenOrders`).

```
1  public class Restaurant {
2      private Set<Order> openOrders = new HashSet<Order>();
3      private int priceOfOpenOrders = 0;
4
5      public static int calculatePrice(List<Order> orders) {
6          int total = 0;
7          for (Order order : orders) {
8              if (order.getPrice() > 500 && order.isMainCourse()) {
9                  total += order.getPrice();
10             }
11         }
12         return total;
13     }
14
15     public void removeOrder(Order orderToRemove) {
16             openOrders.remove(orderToRemove);
17             priceOfOpenOrders -= orderToRemove.getPrice();
18     }
19
20     /*@ requires !openOrders.contains(orderToAdd);
21       @ ensures openOrders.size() == \old(openOrders.size()) + 1;
22       @ ensures openOrders.contains(orderToAdd);
23       @*/
24     public void addOrder(Order orderToAdd) {
25         if (!openOrders.contains(orderToAdd)) {
26             openOrders.add(orderToAdd);
27             priceOfOpenOrders += orderToAdd.getPrice();
28         }
29     }
30 }
31
32 public class Order {
33         public int price;
34         public boolean isMainCourse;
35
36         public int getPrice() {return price;}
37         public boolean isMainCourse() {return isMainCourse;}
38 }
```

(a) Ersetzen Sie den Methodenkörper der `calculatePrice`-Methode durch einen parallelen Stream. *Hinweis:* Zum Aufsummieren von Werten stellt `IntStreams` die `sum()`-Operation bereit. [4 Punkte]

Beispiellösung:

```
return orders.parallelStream().filter(o -> o.getPrice() > 500)
.filter(o -> o.isMainCourse()).mapToInt(Order::getPrice).sum();
```

(b) Schreiben sie eine Vor- und zwei Nachbedingungen für die Methode `removeOrder`, die für den Aufruf erfüllt sein muss bzw. von der Methode erfüllt werden. Die trivialen Vor- und Nachbedingungen `true` und `false` sind keine gültigen Lösungen. Die Größe der Felder hat keine Korrelation zur Länge der Lösung. [3 Punkte]

Beispiellösung: Mögliche Vor- und Nachbedingungen:

```
1 @ requires openOrders != null
2 @ requires openOrders.contains(orderToRemove)
3 @ ensures priceOfOpenOrders = \old(priceOfOpenOrders) - orderToRemove
    .getPrice()
4 @ ensures openOrders.contains(orderToRemove) == false
5 @ ensures openOrders.size() == \old(orders.size()) - 1
```

Gefordert waren lediglich eine Vor- und zwei Nachbedingungen.

(c) Welches Entwurfs-Prinzip bezüglich *Design by Contract* verletzt die `addOrder`-Methode? Wie kann die Methode angepasst werden, damit sie das Prinzip nicht mehr verletzt? [2 Punkte]

Beispiellösung: Die Methode verletzt das Non-redundancy principle. Die Abfrage, ob `openOrders` `orderToAdd` enthält, kann weggelassen werden.

Aufgabe 9 (Grammar Engineering) [8 Punkte]

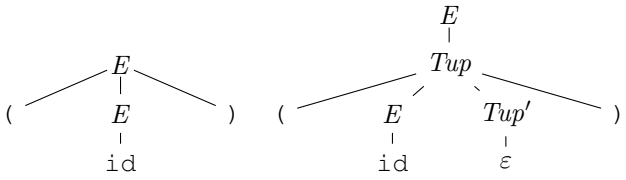
Gegeben sei die folgende Grammatik G mit Startsymbol E und den Terminalen `(`, `)`, `,`, `id`.

$$
\begin{array}{rcl}
E & \rightarrow & \mathit{Tup} \mid \texttt{id} \mid \texttt{(}\ E\ \texttt{)} \\
\mathit{Tup} & \rightarrow & \texttt{(}\ E\ \mathit{Tup}'\ \texttt{)} \\
\mathit{Tup}' & \rightarrow & \texttt{,}\ E\ \mathit{Tup}' \mid \varepsilon
\end{array}
$$

Es handelt sich um eine Sprache mit Bezeichnern, Klammern und Tupeln.

(a) Diese Grammatik ist mehrdeutig. Geben Sie zwei verschiedene Ableitungsbäume für dasselbe Wort in $L(G)$ an. [3 Punkte]

Beispiellösung:



(b) Geben Sie zwei verschiedene Änderungsvorschläge für unterschiedliche Produktionen von G an, die genau eine Produktion betreffen und die die Mehrdeutigkeit beseitigen, ohne $L(G)$ zu verändern. [3 Punkte]

Beispiellösung: Es gibt mehrere Möglichkeiten:

i. Entferne die Produktion $E \rightarrow \texttt{(}\ E\ \texttt{)}$

ii. Verändere die Produktion $E \rightarrow \texttt{(}\ E\ \texttt{)}$ zu $E \rightarrow \texttt{(}\ E\ \mathit{Tup}'\ \texttt{)}$

iii. Verändere die *Tup*-Produktion zu $\mathit{Tup} \rightarrow \texttt{(}\ E\ \texttt{,}\ E\ \mathit{Tup}'\ \texttt{)}$

iv. Verändere die ε-Produktion zu $\mathit{Tup}' \rightarrow \texttt{,}\ E$

(c) G muss nur um eine zusätzliche Produktion ergänzt werden, damit Tupel-Ausdrücke in ihrer letzten Komponente leer sein dürfen. Dann wäre z.B. `(id,id,)` Teil der Sprache, nicht aber `(id,,id)`. Geben Sie diese Produktion an. [1 Punkt]

Beispiellösung: $\mathit{Tup}' \rightarrow \texttt{,}$. Alternativ (aber mehr mehrdeutig): $\mathit{Tup} \rightarrow \texttt{(}\ E\ \mathit{Tup}'\ \texttt{,}\ \texttt{)}$.

(d) Geben Sie eine zusätzliche Produktion für G an, die $L(G)$ um Funktionsaufrufe `id(id,id)` wie z.B. in `(id(id,id),id)` erweitert. [1 Punkt]

Beispiellösung: $E \rightarrow \texttt{id}\ \mathit{Tup}$

Aufgabe 10 (Rekursiver Abstieg) [13 Punkte]

Gegeben sei die folgende SLL(1)-Grammatik G mit Startsymbol A und den Terminalen `(`, `)`, `,`, `id`.

$$
\begin{array}{rcll}
A & \rightarrow & B & \texttt{(} \qquad \mid \texttt{id}\ \texttt{id} \\
B & \rightarrow & \texttt{(}\ A\ D & \texttt{(} \\
D & \rightarrow & \texttt{,}\ L\ \texttt{)} & \texttt{,} \qquad \mid \texttt{)} \quad \texttt{)} \\
L & \rightarrow & A\ R & \texttt{(} \mid \texttt{id} \\
R & \rightarrow & \texttt{,}\ L & \texttt{,} \qquad \mid \varepsilon \quad \texttt{)}
\end{array}
$$

(a) Tragen Sie die Indizmengen aller Produktionen von G oben in die grau hinterlegten Lösungsfelder ein. [4 Punkte]

(b) Geben Sie für jedes Nichtterminal von G einen rekursiven Abstiegsparser an. Brechen Sie bei Parserfehlern mit der globalen `error()`-Funktion ab. [9 Punkte]

Hinweis: Sie können davon ausgehen, dass die Lexer-Schnittstelle wie folgt aussieht:

- Die Token sind Werte eines `enum` namens **`Token`** und heißen **`LP`**, **`RP`**, **`COMMA`**, **`IDENT`** und **`EOF`** für **`(`**, **`)`**, **`,`**, Identifier sowie das Ende der Eingabe. Token können bspw. mit `==` verglichen werden.
- `lexer.current` gibt nichtkonsumierenden Zugriff auf das aktuelle Token.
- `lexer.lex()` konsumiert das aktuelle Token.

```
void expect(Token tok) {
  if (lexer.current != tok) { error(); }
  lexer.lex();
}
```

Beispiellösung:

```
void parseA(){
  switch(lexer.current) {
  case LP:
    parseB();
    break;
  case IDENT:
    lexer.lex();
    break;
  default:
    error();
  }
}

void parseD(){
  switch(lexer.current) {
  case COMMA:
    lexer.lex();
    parseL();
    expect(RP);
    break;
  case RP:
    lexer.lex();
    break;
  default:
    error();
  }
}

void parseB(){
  expect(LP);
  parseA();
  parseD();
}

void parseL(){
  parseA();
  parseR();
}

void parseR(){
  switch(lexer.current) {
  case COMMA:
    lexer.lex();
    parseL();
    break;
  case RP:
    break;
  default:
    error();
  }
}
```